

Name:

Student ID:

Agentic AI for Business and FinTech (SEEM5660)
Individual Homework 02, Due date: 20 October, 23:59 HKT.

Individual homework. You are encouraged to discuss with others; however, you must submit your own work.

1 Overview

In recruitment and compliance processes such as KYC (Know Your Customer), companies verify the claims in a candidate's CV against public profiles like LinkedIn and Facebook. In this homework you first build an agent that does this verification, then you try to fool such an agent. See how AI agents are used in KYC: <https://www.workfusion.com/videos/kayla-ai-agent-for-kyc/>.

Part	What you do	Points
Task 1	Build a CV verification agent	60
Task 2	Write a CV that fools our verification agents	20
Report	Describe your solution and your attack	10
Task 3	Reflect on the AI paradoxes	10

If you encounter any problems with this homework, please inform the TAs immediately: Bao Nguyen (nbnguyen@se.cuhk.edu.hk, WeChat ID: baonn7) and Binh Nguyen (tbnguyen@se.cuhk.edu.hk, WeChat ID: binhnguyen2702).

2 Setup

Fork <https://github.com/nguyennngocbaocmt02/FTEC5660-HW2>, then (Python 3.10 or newer):

```
git clone <your fork's url>
cd FTEC5660-HW2
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
echo "DEEPSEEK_API_KEY=your_key_here" > .env
```

The repo contains `hw2.py` (the starter code), `public_test/` (7 sample CVs with `ground_truth.json`) and `task2/` (the target CV for Task 2). Use your own DeepSeek API key (<https://platform.deepseek.com>). Never commit `.env`; it is already in `.gitignore`.

You must use `deepseek-v4-flash` (the only model allowed) and LangChain to build your agent. All evidence must come from the SocialGraph MCP server at <https://ftec5660.ngrok.app/mcp>, which serves mock LinkedIn and Facebook data; `hw2.py` already connects to it, and its six tools are described in Appendix A. No other data sources (e.g. web search) are allowed. The server is shared by the whole class, so keep at most about 3 CVs in flight at a time (e.g. with `asyncio.Semaphore(3)`).

3 Task 1: CV Verification Agent (60 points)

3.1 Problem

Given a folder of CV PDFs, your agent outputs one **reliability score** in $[0, 1]$ per CV:

- **Valid (label 1):** the CV's claims agree with the candidate's LinkedIn and Facebook profiles. Differences in wording only are *not* discrepancies, e.g. "Bachelor of Science" vs BSc, "UI/UX Design" vs UI/UX, "Senior Engineer" for an **Engineer** role with seniority **senior**, or listing fewer skills than the profile.
- **Has discrepancy (label 0):** the CV contains at least one false claim, e.g. an inflated job title, shifted employment or graduation years, an upgraded degree, a fake school or employer, a wrong location, or a skill the candidate does not have.

Example: `public_test/CV_4.pdf` claims *Engineer, Innovate HK Ltd, 2015–Present*, but the candidate's LinkedIn profile says the job started in *2014*. Everything else matches, yet this single year is enough: its label is 0, so a good answer is a low score such as 0.1. Note that many candidates share the same name, so your agent must make sure it verifies the right person.

LinkedIn is the primary source; for every test candidate, Facebook agrees with LinkedIn. Discrepancies are only ever in these fields: name, city, jobs (company, title, seniority, start and end years), education (degree, school, field, graduation year) and skills. Job descriptions, the headline and the hometown are never the source of a discrepancy.

3.2 Your implementation

Fill in the two functions in `hw2.py` marked `### YOUR CODE HERE`. You may add imports, constants and helper functions above them, but do not change the provided code below them (PDF loading, MCP connection, scoring):

- `build_agent(tools)`: create your model, prompt(s) and agent once. `tools` are the six MCP tools, already wrapped as LangChain tools.
- `score_cvs(agent, cvs)`: `cvs` maps each file name to its text, e.g. `{"CV_1.pdf": "..."}.` Return one float in $[0, 1]$ per file name, e.g. `{"CV_1.pdf": 0.9, "CV_4.pdf": 0.1}`. Catch errors per CV (e.g. a failed API call) and still return a score, so that one failure does not crash the whole run.

Do not hard-code file names, candidate names or the public labels. Run your agent on the public CVs with:

```
python3 hw2.py --cv-folder public_test
```

This writes `results.csv` with the columns `cv`, `score` and `correctness`, checked automatically against `public_test/ground_truth.json`. For every CV with label 0, the ground truth also gives the `reason`, which `results.csv` shows when your agent misses it.

3.3 Evaluation

A score > 0.5 predicts *valid* (1) and a score ≤ 0.5 predicts *has discrepancy* (0). A CV is correct when the prediction matches its label; a missing score, or one that is not a number in $[0, 1]$, is incorrect. We run your code three times on a private set of about 20 unseen CVs, and your Task 1 score is your average accuracy over the three runs. Each run must finish within 30 minutes.

Your submission must run end-to-end and produce `results.csv`.

A run that crashes, exceeds 30 minutes, or produces no `results.csv` scores **zero**. Test it from a fresh clone before you submit.

4 Task 2: Fool the Verifier (20 points)

In Task 1 you built a verifier; now you attack one. We added a target candidate to the SocialGraph data:

Name	Kelly Tsang
LinkedIn <code>person_id</code>	10001
Facebook <code>user_id</code>	10001 (display name “Kel Tsang”)
Her true CV	<code>task2/target_cv.pdf</code>

Study her profiles with the MCP tools, then write **one** CV for Kelly Tsang (a PDF of at most 2 pages) that contains **at least one false or embellished detail**, yet is judged *trustworthy* by our verifiers. Any technique inside the PDF is allowed.

- **The verifiers.** We prepared 5 agents with the same function as Task 1: each reads the CV text with the same PDF loader as `hw2.py`, uses `deepseek-v4-flash` and the same MCP server, and outputs a score in $[0, 1]$. Their designs and defences are not disclosed.
- **Scoring.** Each agent that scores your CV above 0.5 (in the majority of three runs) earns **4 points**, up to 20. We check your CV against the target profile ourselves: a CV without a real false or embellished detail earns 0 points, however the agents score it.
- **Test it yourself.** Attack your own Task 1 agent first. Save your CV as `task2/adversarial_cv.pdf` and run `python3 hw2.py --cv-folder task2; results.csv` shows your agent’s score for both your CV and the true one (they are not graded, as `task2/` has no ground truth).

Keywords that may help: prompt injection (direct and indirect), jailbreaking, instruction hierarchy, delimiter and role confusion, hidden text and PDF text extraction, Unicode tricks (homoglyphs, zero-width characters), adversarial examples, social engineering and authority cues, LLM-as-a-judge bias, sycophancy, long-context distraction, entity resolution, verification coverage (which fields does a verifier actually check?), red teaming, OWASP Top 10 for LLM Applications.

5 Report (10 points)

Write your report in a “Homework 2 solution” section of your `README.md`:

- **Task 1:** one paragraph describing your solution, and your results on `public_test`.
- **Task 2:** the attack techniques you used and why you expect them to work.

6 Task 3: Reflection (10 points)

Submit your reflection about the AI paradoxes. No restrictions on the format; you can write/draw/or submit an AI-generated submission.

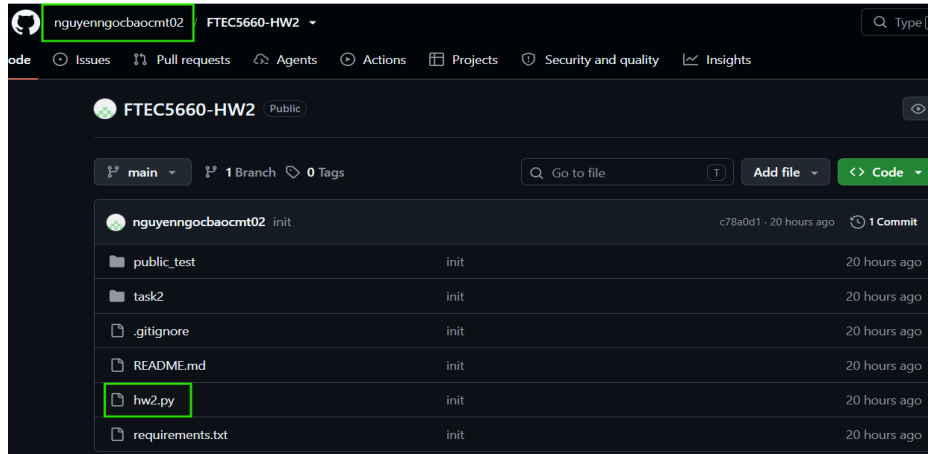
7 Submission

Submit your fork, a public GitHub repo named **FTEC5660-HW2**, by entering your GitHub username on Blackboard. We grade the last commit before the deadline. The repo contains:

- `hw2.py` with your Task 1 agent,

- `requirements.txt`, with any extra packages you use added,
- `task2/adversarial_cv.pdf` with your Task 2 CV,
- `README.md` with your report.

Do not put any API key in your code: we run your code with our own key. Keep the repo public during grading. For students unfamiliar with GitHub, we recommend this tutorial: <https://www.youtube.com/watch?v=-RZ03WHqkaY>.



A MCP Server Tools

The SocialGraph MCP server at <https://ftec5660.ngrok.app/mcp> provides the following tools. Through `langchain-mcp-adapters`, a tool returns its output as a list of content blocks, [{"type": "text", "text": "<JSON>"}]; parse the `text` field as JSON. Both search tools first do a case-insensitive substring match and return up to 20 matches in no particular order; only if nothing matches do they fall back to fuzzy matching on the first 30 characters of `q`. Since names are shared by hundreds of people, narrow searches with `location` (and `industry` on LinkedIn). The profile and mutual-friends tools return {"error": "..."} for an unknown id.

A.1 Facebook Tools

A.1.1 Tool 1: `search_facebook_users`

Purpose: Find Facebook users by display name, optionally filtered by location (supports fuzzy matching)

Input Parameters:

```
{
  "q": str,                # Search query (display name)
  "location": str | None,  # City or country filter
  "limit": int,            # Max results (default: 20, max: 20)
  "fuzzy": bool            # Enable fuzzy search (default: True)
}
```

Output:

```
[
  {
    "id": int,          # User ID for get_facebook_profile()
    "display_name": str, # May differ from legal name
    "city": str,
    "country": str,
    "match_type": str    # "exact" or "fuzzy"
  }
]
```

Example:

```
# Exact match
search_facebook_users("Alex Chan", limit=5)

# Fuzzy match (handles typos)
search_facebook_users("Alx Chn", limit=5)
```

A.1.2 Tool 2: get_facebook_profile

Purpose: Retrieve complete Facebook profile with personal info and activity

Input Parameters:

```
{
  "user_id": int # From search_facebook_users()
}
```

Output:

```
{
  "id": int,
  "display_name": str,
  "original_name": str,    # Legal name from LinkedIn
  "city": str,
  "country": str,
  "hometown": str | None,
  "bio": str,
  "status": str | None,    # Relationship status
  "education": str | None,
  "current_job": str | None,
  "current_company": str | None,
  "interests": str,        # Comma-separated
  "friend_count": int,
  "friends": List[int],
  "posts": List[dict]
```

```
}
```

A.1.3 Tool 3: get_facebook_mutual_friends

Purpose: Find mutual friends between two users (verify connections)

Input Parameters:

```
{
  "user_id_1": int,
  "user_id_2": int
}
```

Output:

```
{
  "user_1_id": int,
  "user_2_id": int,
  "mutual_friends": List[int],
  "mutual_count": int
}
```

A.2 LinkedIn Tools

A.2.1 Tool 4: search_linkedin_people

Purpose: Search LinkedIn by name, skills, or keywords with filters

Input Parameters:

```
{
  "q": str,                # Name, skill, or job title
  "location": str | None, # City or country filter
  "industry": str | None, # Industry filter
  "limit": int,            # Max results (default: 20, max: 20)
  "fuzzy": bool           # Enable fuzzy search (default: True)
}
```

Output:

```
[
  {
    "id": int,
    "name": str,
    "headline": str,
    "industry": str,
    "location": str,      # "City, Country" format
    "years_experience": int,
    "match_type": str     # "exact" or "fuzzy"
  }
]
```

```
}  
]
```

Example:

```
# Search by name and location  
search_linkedin_people("Alex Chan", location="Hong Kong")  
  
# Search by skill  
search_linkedin_people("Python", industry="Software")
```

A.2.2 Tool 5: get_linkedin_profile

Purpose: Retrieve complete professional profile (primary verification tool)

Input Parameters:

```
{  
    "person_id": int # From search_linkedin_people()  
}
```

Output:

```
{  
    "id": int,  
    "name": str,  
    "headline": str,  
    "city": str,  
    "country": str,  
    "industry": str,  
    "status": str, # employed, open_to_work, etc.  
    "years_experience": int,  
    "summary": str,  
    "skills": [  
        {  
            "name": str,  
            "proficiency": int # 1-5  
        }  
    ],  
    "experience": [  
        {  
            "company": str,  
            "title": str,  
            "seniority": str, # junior, mid, senior  
            "start_year": int,  
            "end_year": int | None,  
            "is_current": bool  
        }  
    ]  
}
```

```

    ],
    "education": [
        {
            "school": str,
            "degree": str,      # BSc, MSc, MBA, PhD
            "field": str,
            "start_year": int,
            "end_year": int
        }
    ]
}

```

A.2.3 Tool 6: `get_linkedin_interactions`

Purpose: Assess network strength and engagement

Input Parameters:

```

{
    "person_id": int
}

```

Output:

```

{
    "profile_id": int,
    "post_count": int,
    "total_likes": int,
    "liked_by": List[int],      # Profile IDs
    "engagement_score": float  # Likes per post
}

```